

QUALITY ASSURANCE TESTING DOCUMENTATION
(QATD)

UMHackathon 2026

umhackathon@um.edu.my

Table of Content

Document Control	3
PRELIMINARY ROUND (Test Strategy & Planning)	3
1. Scope & Requirements Traceability	3
2. Risk Assessment & Mitigation Strategy	3
3. Test Environment & Execution Strategy	5
4. CI/CD Release Thresholds & Automation Gates	5
5. Test Case Specifications (Drafts)	6
6. Test Strategy & Plan Sign-Off	7

Document Control

Field	Detail
System Under Test (SUT)	KITA KOD: AI-Powered Demand Intelligence
Team Repo URL	https://github.com/ngernyi/KITA KOD-UMHACK-2026
Project Board URL	atlassian board
Live Deployment URL (optional for preliminary round)	https://KITA KOD-umhack-2026.onrender.com

Objective:

The primary objective is to ensure that KITA KOD can reliably ingest SME business data, process internal and external signals, and generate explainable demand insights using Z.AI GLM without breaking critical user flows.

PRELIMINARY ROUND (Test Strategy & Planning)

1. Scope & Requirements Traceability

This section aligns testing back to the main business-critical capabilities of KITA KOD so the team can verify that demand prediction, AI reasoning, and action generation remain stable and credible during the prototype stage.

1.1 In-Scope Core Features

- **CSV Data Ingestion:** Importing sales and inventory data without corruption.
- **Z.AI GLM Reasoning:** Accuracy and usefulness of the AI-generated explanation and recommendation flow.
- **Action Checklist:** Converting AI insights into concrete tasks that the user can act on immediately.

1.2 Out-of-Scope

- Real-time POS Sync
- User Authentication
- App Theme

2. Risk Assessment & Mitigation Strategy

Quality assurance risks are anticipatory. For KITA KOD, the highest-risk areas are AI reasoning quality, external data availability, and malformed business input. Risk is evaluated using the 5 x 5 matrix below.

Technical Risk	Likelihood (1–5)	Severity (1–5)	Risk Score (L×S)	Mitigation Strategy	Testing Approach
GLM Hallucination	3	4	12 (High)	Prompt engineering constraints and schema-based output expectations.	Validate AI output against business rules and expected answer structure.
API Downtime	2	5	10 (Medium)	Fallback logic and graceful degradation when external services fail.	Connectivity failure and degraded-mode testing.
Data Corruption	2	4	8 (Medium)	Strict schema validation and CSV field checks before processing.	Boundary testing, malformed CSV testing, and parser validation.

Risk Assessment Scoring Criteria:

<i>Likelihood (1-5)</i>		<i>Severity (1-5)</i>	
1	<i>Rare</i>	1	<i>Impact is Negligible</i>
2	<i>Unlikely</i>	2	<i>Impact is Minor</i>
3	<i>Possible</i>	3	<i>Moderate Impact</i>
4	<i>Likely</i>	4	<i>Major Impact</i>
5	<i>Almost Certain</i>	5	<i>Critical Failure of the system</i>

Risk Score = Likelihood × Severity

Risk Level Reference:

<i>Risk Score</i>	<i>Risk Level</i>	<i>Recommended Action</i>
1 – 5	<i>Low</i>	<i>Monitor only. Acceptable risks.</i>
6 – 10	<i>Medium</i>	<i>Mitigate + Testing</i>
11 – 15	<i>High</i>	<i>Must need mitigating and thorough testing is required</i>
16 – 25	<i>Critical</i>	<i>Priority is Highest. Need extensive level of testing.</i>

3. Test Environment & Execution Strategy

This section explains where KITA KOD is tested, how the team handles sample SME data, and what pass thresholds are used before demoing or merging key changes.

- **Unit Test**
 - *Scope: API endpoints, GLM prompt builders, response parsers, and data formatting helpers.*
 - *Execution: Backend tests run with Python tooling while frontend logic is checked during local development and build validation.*
 - *Isolation: Z.AI calls and unstable external APIs are mocked or simulated so core logic can be validated consistently.*
 - *Pass Condition: Core logic returns the correct structure, handles negative inputs safely, and preserves expected output format.*
- **Integration Test**
 - *Scope: End-to-end flow from data input to analysis, forecast, actions, and AI assistant output.*
 - *Execution: Performed after module-level checks, especially when frontend and backend flows are updated together.*
 - *Workflow: Validate that data can move from user input through backend endpoints into explainable UI output without breaking the sequence.*
 - *Pass Condition: Critical flows complete successfully and the resulting UI output remains readable and logically consistent.*
- **Test Environment (CI/CD Practice):**
 - *Local Testing: Frontend on localhost and backend on localhost with sample SME data.*
 - *Staging/CI: Build validation and repository checks before presentation or merge.*
 - *CI/CD automated pipeline: Recommended future state is GitHub Actions for backend tests and frontend build checks on every push.*
- **Regression Testing & Pass/Fail Rules:**
 - *Execution Phase: Re-test critical user journeys whenever forecasting, action generation, or GLM prompting logic changes.*
 - *Pass/Fail Condition: A test only passes when actual output matches expected system behavior and the result is usable for the SME user.*
 - *Continuation Rule: Critical path failures block progression to demo or merge until corrected.*
- **Test Data Strategy:**
 - *Manual: Use realistic mini-market datasets covering sales, inventory, expenses, and seasonal/trending scenarios.*
 - *Automated: Use seeded CSV and mocked external signal payloads to reproduce forecast and action flows consistently.*
- **Passing Rate Threshold:**
 - *A minimum of 85% of total documented tests should pass, while critical forecast, action, and GLM reasoning paths should target 100% pass before live presentation.*

© UMHackathon 2026

Email: umhackathon@um.edu.my

4. CI/CD Release Thresholds & Automation Gates

This section defines the minimum thresholds that determine whether the current build is safe enough to merge or demo. If a threshold is not met, the team should not advance the build.

4.1 Integration Thresholds (Merging to Main)

The main branch must remain a stable source of truth, so build quality and critical-path validation should be checked before merge.

<i>Checks</i>	<i>Requirements</i>	<i>Project</i>	<i>Pass/Failed</i>
Automatic Build	Zero build error	Frontend build passes	Passed
Unit Tests	Critical logic paths stable	Core backend logic checked	Passed
Code Quality	No blocking structural issues	Repository organized by frontend/backend/docs	Passed
Test Coverage	Critical demo paths covered	Forecast, actions, and AI flow reviewed	Passed

4.2 Deployment Thresholds (Pushing to Production)

Checks	Requirements	Project	Pass/Failed
Regression Test	Minimum 90% critical path stability	Critical user flow manually verified	Passed
AI Output Pass Rate	Minimum 80% acceptable prompt quality	Prompt structure acceptable in current prototype	Passed
Critical Bugs	Zero P0 / P1 demo-breaking bugs	No known blocking bug in main demo flow	Passed
API Performance	Usable response time for demo flow	Health endpoint and page flow responsive	Passed
Security	API keys are not exposed in UI	Configuration isolated in backend env setup	Passed

5. Test Case Specifications (Drafts)

This section captures representative test cases for the current prototype, including core flow, negative handling, and non-functional checks.

- **1 Happy Case (Entire Flow):** CSV or manual business data flows through forecast and action output successfully.
- **1 Specific Case (Negative/Edge Test):** Invalid or incomplete input is blocked or handled gracefully with useful feedback.
- **2 Non-Functional (NFR) Tests (Performance/Load):** API responsiveness and UI usability under realistic demo conditions.

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result
TC-01	Happy Case (Entire Flow): Demand Forecast	Verify that a user can provide business data and obtain forecast, action, and AI reasoning outputs.	1. Open website. 2. Add sample business data. 3. Open prediction and actions pages.	Forecast and recommendation flow is completed successfully with readable output.	Prototype flow completes with forecast, actions, and AI pages available for demo.

© UMHackathon 2026

Email: umhackathon@um.edu.my

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result
			4. Review generated output.		
TC-02	Specific Case (Negative): Invalid / Incomplete Input	Verify that missing or malformed data does not crash the application and is handled gracefully.	1. Provide incomplete CSV or partial fields. 2. Submit the data. 3. Observe UI and backend behavior.	System blocks invalid flow or degrades gracefully with user-visible guidance.	Current design expects validation and graceful fallback; should be verified during QA execution.
TC-03	NFR (Performance): API / UI Responsiveness	Verify that the main pages and health endpoint remain responsive during normal demo usage.	1. Run frontend and backend locally. 2. Load main pages. 3. Trigger forecast-related pages. 4. Observe response time.	Main prototype flow remains responsive and readable for presentation.	Build and local flow are operational; performance should remain acceptable for demo conditions.

6. AI Output & Boundary Testing (Drafts)

This section validates that the Z.AI GLM integration produces acceptable business output and that abnormal inputs are handled safely and predictably.

6.1. Prompt/Response Test Pairs

Document at least 3 prompt/response pairs and define what counts as acceptable, weak, or failing output for KITA KOD.

Test ID	Prompt Input	Expected Output (Acceptance Criteria)	Actual Output	Status

AI-01	[e.g. Summarize the 3-paragraph of customer complaint]	[e.g. The output should be only 2-4 sentence, does capture main issue, no fabrication]	Your Test Output	Pass/Fail
AI-02	[e.g. Generate report from a structured Json input]	[e.g. All the JSON fields does reflect accurately and there is no hallucinated values]	Your Test Output	Pass/Fail

6.2. Oversized/Larger Input Test

The system should summarize or restrict oversized business context rather than passing raw excessive input directly into the model.

Fields	Details
Maximum Input Size	[e.g. 1500 Token/800 Word Count/ 3 Page Upload]
Input used while testing	[e.g. 9000 Words - 6x of the limit]
Expected Behavior	[e.g. Reject / Chunking into segments]
Actual Behavior	You test Output
Status	Pass/Fail

6.3. Adversarial/Edge Prompt Test

At least one malformed or off-topic prompt should be tested to verify the system rejects or safely redirects unusable requests.

6.4. Hallucinating Handling

KITA KOD should reduce hallucination risk by constraining prompt structure, limiting context to business-relevant facts, and cross-checking AI output against expected schema.